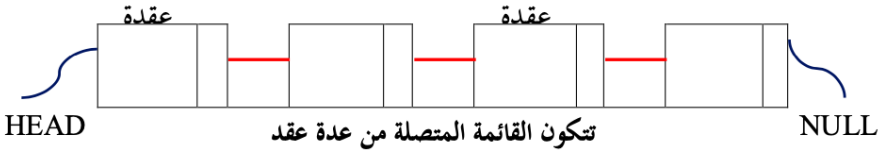
القوائم المتصلة Linked Lists

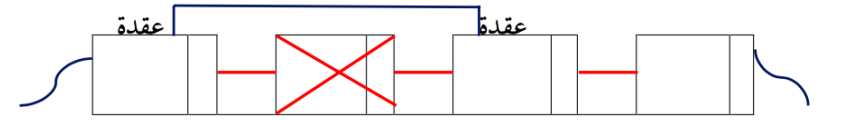
عندما تحدثنا عن المصفوفات ذكرنا بان المصفوفة عبارة عن منطقة في الذاكرة تتكون من عدد محدد ومتجانس من المواقع المتجاورة وذكرنا بان المشكلة الرئيسية في المصفوفة أنها غير مرنة، بمعنى أنها ليست قابلة للزيادة أو النقصان وقت تشغيل البرنامج .هنا يأتي دور القائمة المتصلة لحل هذه المشكلة.

تعريف القائمة المتصلة:

هي عبارة عن هيكل (تركيب) بيانات مكون من عقد nodes مرتبطة مع بعضها البعض، ولها بداية ونهاية.



تتكون القائمة المتصلة من عدة عقد

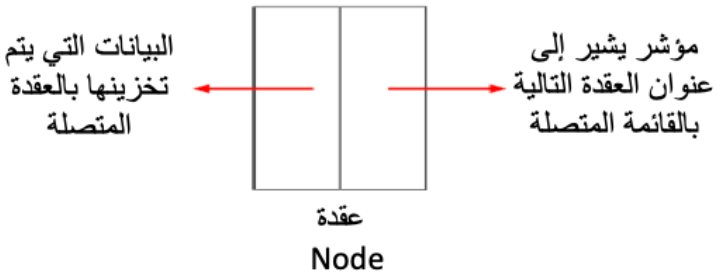


تتميز القائمة المتصلة بالقدرة على حذف العناصر أثناء تشغيل البرنامج

ما هي العقدة؟

العقدة هي عنصر في القائمة المتصلة وتتكون القائمة المتصلة عادة من مجموعة من العقد. مقارنة مع المصفوفة، تتكون المصفوفة من مجموعة عناصر في حين تتكون القائمة المتصلة من مجموعة عقد كل عنصر في المصفوفة ومكان لتخزين قيمة بيانية معينة كذلك العقدة في القائمة، تستخدم لتخزين قيمة بيانية معينة أيضاً. طبعاً مع وجود بعض الاختلافات بين العقدة والعنصر من حيث الهيكلية وكذلك طريقة الوصول إلى – والتعامل مع كل منهم. وتتكون كل عقدة من:

1. حقل البيانات المراد تخزينها بالعقدة.
2. حقل المؤشر والذي يشير إلى العقدة التالية.



مع العلم بأنه يمكن أن يكون بالعقدة الواحدة أكثر من حقل واحد لتخزين البيانات – أي تحتوي على أكثر من قيمة بيانية واحدة بمؤشر واحد يشير إلى العقدة التالية.

وللقوائم المتصلة Linked Lists عدة استخدامات مهمة في نظم تشغيل الحواسيب نظراً لمرونتها في التعامل مع الذاكرة والبيانات بطريقة ديناميكية مما يجعلها أداة قوية في تصميم وتنفيذ أنظمة التشغيل المعقدة. بعض الاستخدامات الرئيسية تشمل:

1. **إدارة الذاكرة:(Memory Management)**

في أنظمة التشغيل، يمكن استخدام القوائم المتصلة لتنظيم الذاكرة وتخصيصها بفعالية. حيث يمكن استخدامها لتمثيل الكتل المخصصة في الذاكرة، حيث تمثل كل عقدة في القائمة كتلة ذاكرة تم تخصيصها. عند تحرير الذاكرة، يمكن إزالة العقدة المرتبطة بالموقع المحدد، مما يوفر آلية مرنة للتعامل مع الذاكرة.

2. **إدارة العمليات:(Process Management)**

- تستخدم القوائم المتصلة لإدارة قائمة العمليات في نظام التشغيل. فكل عملية في النظام يمكن تمثيلها بعقدة في القائمة، وتحتوي العقدة على معلومات مثل حالة العملية، الأولوية، معرف العملية (PID) ، والموارد المستخدمة.
- القائمة المتصلة تمكن النظام من إضافة أو إزالة العمليات بسهولة، خاصة في الحالات التي تتطلب إضافة أو حذف عمليات بشكل مستمر.

3. **جدولة العمليات:(Process Scheduling)**

يمكن استخدام القوائم المتصلة لتمثيل صفوف الانتظار لجدولة العمليات، حيث يتم تنظيم العمليات التي تنتظر الحصول على وحدة المعالجة المركزية أو الموارد الأخرى. على سبيل المثال، يمكن إنشاء قائمة مرتبطة لتمثيل العمليات في حالة الانتظار، ويمكن إضافة أو إزالة العمليات من هذه القائمة حسب الأولوية أو مدة الانتظار.

4. **إدارة الموارد:(Resource Management)**

في أنظمة التشغيل التي تدير العديد من الموارد، مثل الطابعات أو الأقراص، يمكن استخدام القوائم المتصلة لتمثيل الموارد التي يتم استخدامها. يمكن تتبع حالة كل مورد وتحديد أي العمليات تتطلب هذا المورد باستخدام القوائم المتصلة.

5. إدارة الإدخال/الإخراج:(I/O Management)

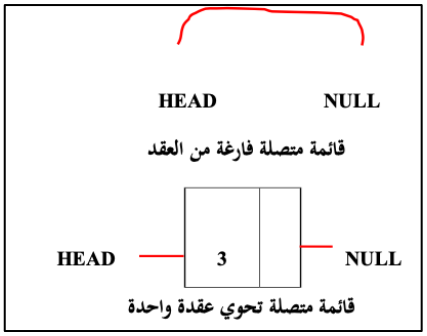
عند التعامل مع ملفات أو تدفقات بيانات كبيرة، يمكن استخدام القوائم المتصلة لتنظيم البيانات المتدفقة من الأجهزة المختلفة. على سبيل المثال، عندما تتم معالجة ملفات أو بيانات في النظام، يتم تنظيم البيانات في شكل قوائم متصلة لتسهيل إضافة أو إزالة البيانات بطريقة مرنة.

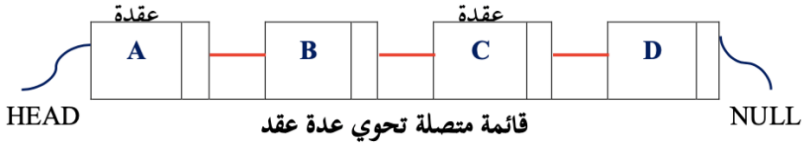
6. نظام الملفات:(File Systems)

في بعض أنظمة الملفات، يتم استخدام القوائم المتصلة لتمثيل الكتل المتسلسلة للبيانات في الملف، مما يساعد في تتبع مكان وجود البيانات في الذاكرة أو على القرص الصلب. يمكن أن تكون الملفات الكبيرة موزعة عبر عدة كتل غير متجاورة، ويمثل كل عقدة في القائمة كتلة من البيانات في الملف.

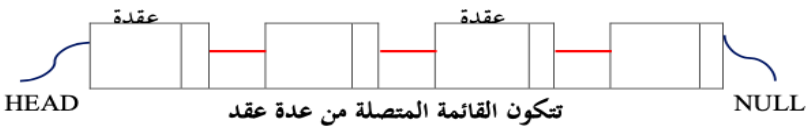
بداية ونهاية القائمة المتصلة

يستخدم المؤشر والذي عادة ما يسمى head كمؤشر لأول عقدة في القائمة المتصلة، وأول عقدة تشير إلى الثانية، والعقدة الثانية تشير إلى العقدة الثالثة،.... وهكذا حتى نصل إلى عقدة تشير إلى NULL والتي تكون هي العقدة الأخيرة في القائمة. وإذا كانت القائمة خالية (فارغة من العقد) فإن هذا المؤشر سيشير إلى NULL مما يعني أن هذا المؤشر يشير إلى لا شيء.





والمشكلة التي قد تحدث أنه لو حذف المؤشر head أو تبدلت قيمته فإننا لانستطيع التعامل مع القائمة وستحدث مشكلة تسرب الذاكرة. لذلك يجب على المبرمج أن يتذكر قيمة المؤشر head بحيث يمكنه الوصول إلى بداية القائمة المتصلة في أي وقت لغرض إجراء العمليات المختلفة عليها مثل العمليات الحذف والإضافة والبحث في القائمة.



تتكون القائمة المتصلة من عدة عقد

لاحظ أن المؤشر HEAD أصبح يشير إلى شيء أخر

يوجد عدة أنواع من القوائم المتصلة Linked Lists التي يمكن استخدامها بناءً على متطلبات النظام أو التطبيق. وهذه الأنواع هي:

1. القائمة الموصولة أحادية الاتجاه:(Singly Linked List)

في هذا النوع من القوائم، يحتوي كل عنصر (عقدة) على بيانات ورابط يشير إلى العنصر التالي في القائمة. ومن خصائصها:

- يمكن الوصول إلى العناصر فقط من البداية نحو النهاية.
- لا يوجد رابط إلى العقد السابقة.

```
struct node {
    int number;
    node* next;
};
typedef node *node_ptr;
```

2. القائمة المتصلة ثنائية الاتجاه:(Doubly Linked List)

في هذه القائمة، تحتوي كل عقدة على رابطين أحدهما يشير إلى العقدة التالية والآخر يشير إلى العقدة السابقة. ومن خصائص هذا النوع من القوائم أنه:

- يمكن التنقل في القائمة في كلا الاتجاهين – من البداية إلى النهاية والعكس.
- الوصول إلى العقد السابقة أسهل من القائمة أحادية الاتجاه.

```
struct node {
    int number;
    node* next;
    node* prev;
};
typedef node *node_ptr;
```

3. القائمة المتصلة الدائرية أحادية الاتجاه:(Circular Singly Linked List)

في هذا النوع من القوائم، يكون العنصر الأخير في القائمة مرتبطًا بالعنصر الأول في القائمة، بحيث يشكل شكل دائري, من خصائصها:

- لا يوجد عنصر "نهاية" ثابتة، حيث أن العقدة الأخيرة تشير إلى العقدة الأولى.
- يمكن التنقل في القائمة من البداية إلى النهاية، ولكن عند الوصول إلى النهاية، يعود المؤشر إلى البداية.

```
struct node {
    int number;
    node* next;
};
typedef node *node_ptr;
```

4. القائمة المتصلة دائرية ثنائية الاتجاه:(Circular Doubly Linked List)

هذه القائمة تجمع بين خصائص القائمة الموصولة الدائرية والقائمة الثنائية الاتجاه. حيث تحتوي كل عقدة على رابطين، أحدهما يشير إلى العقدة التالية، والآخر يشير إلى العقدة السابقة. العقدة الأخيرة تشير إلى العقدة الأولى، والعقدة الأولى تشير إلى العقدة الأخيرة. من خصائصها:

- التنقل في كلا الاتجاهين – من البداية إلى النهاية والعكس.
- التنقل عبر القائمة لا يتوقف بعد الوصول إلى نهاية أو بداية القائمة، لأنها دائرية.

```
struct node {
    int number;
    node* next;
    node* prev;
 };
typedef node *node_ptr;
```

فيما سبق تم تعريف node من النوع struct وتحتوي على متغيرين، الأول من النوع الصحيح واسمه number لتخزين بيانات من النوع الصحيح، والمتغير الثاني اسمه next أو prev حسب نوع القائمة وهو من النوع مؤشر وله نفس تركيبة العقدة node ويستخدم كمؤشر إلى العقد الأخرى.

نلاحظ أنه بعد التركيبة struct قمنا باستخدام الأمر typedef لتعريف نوع بيانات من نوع آخر وهو node_ptr * من نوع التركيبة node حيث يستخدم للإعلان عن عقد جديدة كما سنرى في الأمثلة عند تنفيذ العمليات المختلفة على العقد المتصلة.

الأمر typedef يستخدم لتعريف نوع من نوع آخر وفقاً للصيغة:

Typedef	النوع الأصلي	النوع الجديد
	node	*node_ptr

العمليات على القوائم المتصلة:

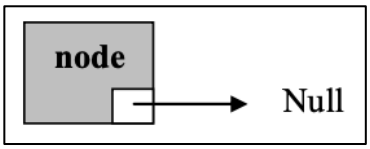
هناك العديد من العمليات التي يمكن اجراؤها على القائمة مثل الإضافة – التعديل – الحذف – عرض العناصر –البحث. نبدأ أولاً بالإضافة ويوجد أربعة طرق لإضافة عناصر إلى القائمة المتصلة وهي:

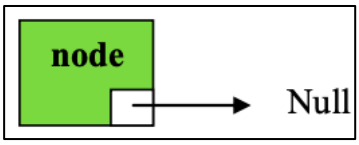
1. إضافة أول عقدة
2. الإضافة إلى الجهة اليمنى (نهاية القائمة)
3. الإضافة إلى الجهة اليسرى (بداية القائمة)
4. الإضافة إلى وسط القائمة

أولاً/ إضافة أول عقدة: بمعنى أنه لا توجد قائمة، ونريد إنشاء قائمه عن طريق إنشاء أول عقدة بها. وحيث أننا قمنا بإنشاء تركيبة لكل عقدة في الخطوات السابقة وتحتوي كل عقدة على حقل البيانات number وحقل المؤشر.

```
node_ptr make_new_node(int x)
{
  node_ptr p;
  p=new node;
  p->number =x;
  p->next =NULL;
  return p;          // p= the address of first node in the list
}
```

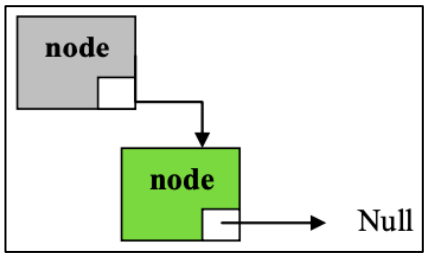
أصبح لدينا الآن عقدة ومؤشرها يشير إلى NULL





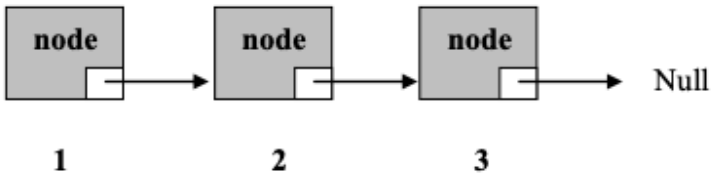
ثانياً/ الإضافة من الجهة اليمنى (نهاية القائمة):

- ننشئ عقدة جديدة ونجعلها تؤشر إلى NULL كما في الخطوة السابقة.
- نجعل مؤشر العقدة الأولى يشير إلى العقدة الجديدة بدلاً من NULL




```
void insert_last(node_ptr &first)
{
   int x;
   cout<<"Enter value in new node \n"; cin>>x;
   node_ptr p,q;
   if (first==NULL)
       first=make_new_node (x);
   else
      {
        p=first;
        while (p->next!=NULL)
             p=p->next ;
       q=make_new_node (x);
       p->next =q;
     }
}
```

لنفترض وجود قائمة تحتوي على 3 عقد ونريد الوصول إلى آخر عقدة، عندها سنتنقل عبر عقد القائمة حتى نصل إلى العقدة التي تؤشر إلى NULL. الآن لنفترض فعلا وجود هذه القائمة التالية:



أولاً، سنجعل p=first حيث first هو عنوان القائمة، أي عنوان أول عقدة فيها، وبذلك أصبح المتغير p يحمل عنوان أول عقدة في القائمة.

ثانياً، نستخدم حلقة التكرار

```
while(p->next != NULL)
p=p->next;
```

كرر طالما أن next لا يساوي NULL سنطبق هذا المثال على القائمة التي افترضناها:

p=1 p->next=2

ثم

p=2 p->next=3

ثم

p=3 p->next=NULL

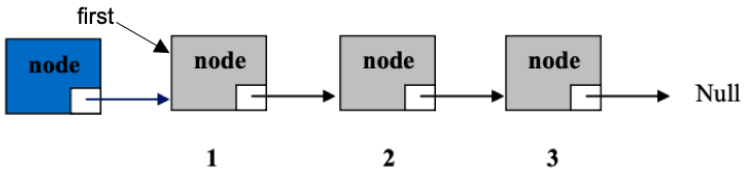
طالما أن p->next أصبحت تساوي NULL، تنتهي بذلك حلقة التكرار ونكون قد وصلنا إلى العقدة الأخيرة. بعد ذلك p->next يشير إلى العقدة الجديدة q، وبذلك يتم إضافة هذه العقدة إلى نهاية القائمة المتصلة.

ثالثاً/ الإضافة من اليسار (بداية القائمة):

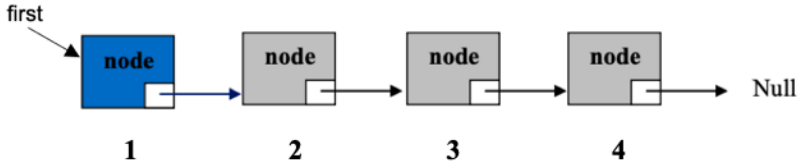
هنا الوضع مختلف قليلاً وأسهل بكثير، وتقوم الفكرة على إنشاء عقدة جديدة وجعلها تؤشر إلى أول عقدة في القائمة، ثم نجعل عنوان القائمة هو العقدة الجديدة.

```
void insert_begin (node_ptr &first)
   {
     int x;
     cout<<"Enter value in new node\n";
     cin>>x;
     node_ptr p;
     p=make_new_node(x);
     p->next=first;
     first=p;
   }
```

قمنا بالتصريح عن p والتي من خلالها نمثل العقدة الجديدة. ثم بعد ذلك نجعل مؤشر هذه العقدة يشير إلى أول عقدة بالقائمة عن طريق الأمر p->next=first;.



نقوم بعد ذلك بجعل مؤشر بداية القائمة first يشير إلى العقدة الجديدة وبذلك يعود هذا المؤشر للإشارة إلى أول عقدة بالقائمة.

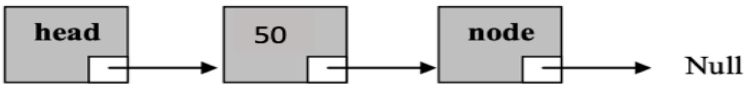


رابعاً: الإضافة من الوسط:

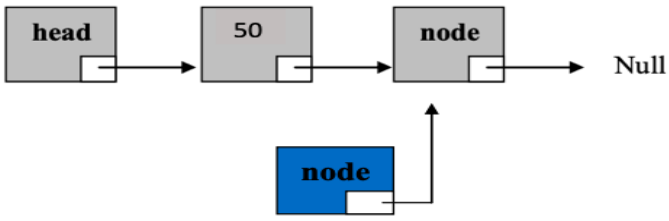
تكمن الفكرة في أن نعثر عن قيمة تدل على العقدة التي نريد أن نضيف عقدة جديدة بعدها. ثم بعد ذلك نضيف بينها وبين العقدة التي تليها عقدة جديدة ونقوم بتعديل المؤشرات كما في المثال التالي:

مثال:

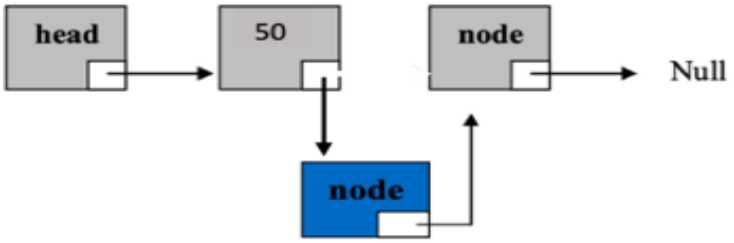
إذا كان لدينا قائمة متصلة من العقد، نبحث عن العقدة التي نريد أن نضيف عقدة جديدة بعدها. ولتكن مثلاً العقدة التي تحتوي على القيمة 50



ننشئ عقدة جديدة ونجعلها تؤشر إلى العقدة التي تلي العقدة المقصودة .



نجعل العقدة المقصودة تؤشر إلى العقدة الجديدة .




```
void insert_middle(node_ptr &first, int n)
{
     int x;
     cout<<"Enter value in new node \n";
     cin>>x;
     node_ptr p,q;
     if(first==NULL)
       cout<< "List not reached……";
     else
       {
         p=first;
         while(p->number!=n && p->next!=NULL)
            p=p->next;
         q=make_new_node (x);
         q->next=p->next;
         p->next =q;
       }
```